

School of Computer Science and Engineering

Faculty of Engineering

The University of New South Wales

Scheduling and Rostering Challenges with LLMs

by

Krishna Mittal

Thesis submitted as a requirement for the degree of

Bachelor of Engineering in Computer Engineering

Submitted: August 2026

Supervisor: Prof. Flora Salim

Student ID: z5625451

Abstract

Healthcare staff rostering is a complex and time-consuming task that must balance an adequate number and skill-mix of healthcare professionals, such as doctors, nurses, allied health staff, and support staff, available at all times to meet the needs of patients and legislated requirements. Traditional scheduling algorithms can manage hard constraints but struggle with dynamic and human-centered nature of scheduling.

This thesis investigates an agentic approach to nurse rostering under the constraints of the Second International Nurse Rostering Competition (INRC-II). The work focuses on the two stages of the problem: first, building a roster satisfying all hard constraints and second modifying this feasible roster through minimization of soft constraint violations. A deterministic roster construction and validation framework was developed to generate a hard-feasible initial solution using the INRC-II problem requirements and the official validator.

The finding demonstrate that the generation of a hard-feasible roster, and the subsequent enhancement of soft-constraint quality, are two different kinds of problems altogether. They also contend that rather than being direct producers of low-level roster changes, LLMs are better suited as high-level search and strategy controllers. The thesis therefore contributes both a validated framework for constructing hard-feasible INRC-II rosters and an experimental evaluation of different ways in which LLM-based agents can be integrated with deterministic optimisation for constrained nurse rostering.

Acknowledgements

I sincerely thank my supervisor, Prof. Flora Salim, for her guidance, expertise, and valuable feedback throughout this thesis. I am also grateful to Dr. Wei Shao for his continued support, ideas, and advice, which helped shape the direction of this work. Finally, I would like to thank my family, friends, and everyone close to me for their encouragement and support throughout this journey.

Abbreviations

LLM Large Language Model

NRP Nurse Rostering Problem

INRC-II Second International Nurse Rostering Competition

CP-SAT Constraint Programming with Boolean Satisfiability

OR Operations Research

JSON JavaScript Object Notation

ReAct Reasoning and Acting

Contents

1 Introduction **1**

2 Literature Review **5**

2.1 Problem Context: Nurse Rostering in Modern Healthcare 5

2.2 Classical and Hybrid Solution Methodologies for NRP 6

2.3 Machine Learning and Reinforcement Learning for Personnel Scheduling 7

2.4 LLMs for Optimization and Operations Research 8

2.4.1 LLMs as Optimizers via Prompt-Driven Search 9

2.4.2 LLMs as Evolutionary Hyper-Heuristics 10

2.4.3 LLM Code Generation and Algorithmic Improvement 12

2.4.4 LLMs for Formal Optimization Modelling 12

2.4.5 LLMs as End-to-End Combinatorial Optimizers 14

3 Problem Definition and INRC-II Constraints **15**

3.1 Nurse Rostering Problem Formulation 15

3.2 INRC-II Problem Structure 16

3.3 Hard Constraints 16

3.3.1 H1 — Single Assignment Per Day 16

3.3.2 H2 — Under-staffing 17

3.3.3 H3 — Shift Type Successions 17

3.3.4 H4 — Required Skills 18
3.4 Soft Constraints 18
3.4.1 Objective Function and Roster Cost 19
3.5 Feasibility and Validation 20

4 Methodology **22**
4.1 Overview 22
4.2 Hard-Feasible Roster Construction 22
4.3 Roster Improvement Framework 24
4.4 Roster Improvement Methods 25
4.4.1 Deterministic Single-Move Search 25
4.4.2 Direct LLM Improvement 26
4.4.3 ReAct Direct Repair 26
4.4.4 ReAct Strategy-Neighbourhood Repair 27
4.4.5 SOCIA-Based Repair 27
4.4.6 Agentic Block Repair 28
4.4.7 Operator-Based Search 28
4.4.8 Compound Repair 29
4.5 Prompt Construction and Search History 30
4.6 Deterministic Candidate Evaluation 31

5 Results **32**
5.1 Experimental Setup 32
5.2 Hard-Feasible Roster Result 33
5.3 Initial Soft-Constraint Penalty 33
5.4 Comparison of Roster-Improvement Methods 33
5.5 Best Results for the 5-Nurse Instance 34

5.6 Results and the 21-Nurse Instance 35

5.7 Summary of Results 35

6 Discussion **36**

6.1 Research Questions 36

6.1.1 R1: Can a systematic computational framework construct a nurse roster that satisfies all INRC-II hard constraints? 36

6.1.2 R2: Can deterministic and LLM-assisted repair strategies reduce the soft-constraint penalty of a roster that is already hard-feasible? 37

6.1.3 R3: How does the role assigned to the LLM affect the effectiveness of roster improvement? 38

6.2 Effect of Problem Size 39

6.3 Effect of Search Neighbourhood 40

6.4 Interaction Between Soft Constraints 40

6.5 Limitations 41

7 Conclusion **43**

7.1 Contributions 44

Appendix 1 **48**

A.1 Data Parsing Prompt 48

A.2 CP-SAT Prompt 51

A.3 Direct LLM Method Prompt 59

List of Figures

2.1 An overview of the OPRO framework 9

2.2 An illustration of ReEvo 11

2.3 An overview of LLMFP 13

List of Tables

3.1 Soft constraints and penalty weights 19

5.1 Large Language Model Configurations Used in the Experiments 32

5.2 Initial Soft-Constraint Penalties for the 5-Nurse and 21-Nurse Instances 33

5.3 Comparison of roster-improvement methods across the three LLM-generated hard-feasible rosters for 5 nurse instance. 34

5.4 Best roster-improvement results for the 5-nurse instance. 34

Chapter 1

Introduction

Effective nurse scheduling is essential for maintaining adequate staffing levels while satisfying operational requirements, contractual provisions, and individual staff preferences. The Nurse Rostering Problem involves assigning nurses to shifts across a specified planning horizon while satisfying a set of interrelated constraints. These constraints are generally divided into hard constraints, which must be satisfied for a roster to be feasible, and soft constraints, whose violation is allowable but associated with penalties in the objective function. Consequently, the objective extends beyond the construction of a feasible roster to include the minimisation of the total penalty arising from soft-constraint violations.

The NRP is a combinatorial optimisation problem whose search space expands substantially with increases in the number of nurses, shifts, planning days, skill requirements, and constraints. As the problem size increases, evaluating potential assignments and their interactions becomes increasingly difficult. An assignment that appears locally beneficial may adversely affect shift coverage, consecutive working patterns, weekend requirements, contractual limits, or other characteristics of the roster. The construction of a feasible solution and the subsequent improvement of its quality should therefore be regarded as related but distinct optimisation tasks.

Traditional approaches to nurse rostering include mathematical programming, con-

straint programming, local search, metaheuristics, and hybrid optimisation techniques. These methods typically rely on explicitly defined search operators, objective functions, and constraint-handling procedures. More recently, Large Language Models have demonstrated capabilities in reasoning, planning, tool use, and sequential decision-making, generating interest in their application to combinatorial optimisation. Beyond functioning as standalone optimisers, LLMs can operate as agents that interpret a problem state, select actions or strategies, receive feedback from an external environment, and incorporate that feedback into subsequent decisions.

However, applying an LLM directly to nurse rostering presents significant challenges. The model must reason about numerous interdependent constraints while maintaining consistency over a lengthy sequence of assignment decisions. Furthermore, an LLM cannot inherently guarantee that a proposed modification satisfies the formal constraints of the problem or improves the objective value. Deterministic validation is therefore necessary when LLM-based reasoning is incorporated into a constrained optimisation framework.

This thesis investigates a hybrid agentic approach to nurse rostering based on the problem formulation of the Second International Nurse Rostering Competition. The research comprises two principal stages. First, a roster-construction procedure is developed to generate a solution satisfying all hard constraints. Once hard feasibility has been established, the second stage examines whether the roster's soft-constraint penalty can be reduced through alternative search and repair mechanisms.

Rather than focusing on a single form of LLM integration, this research examines different distributions of responsibility between the LLM and deterministic optimisation components. The approaches investigated include deterministic single-move search, direct LLM-generated roster modifications, ReAct-style repair, strategy and neighbourhood selection, structured agentic repair, operator-based search, and compound multi-move modifications. In some approaches, the LLM proposes roster modifications directly; in others, it operates at a higher level by selecting repair strategies or neighbourhoods, while deterministic Python procedures generate and evaluate the resulting

candidate solutions. Throughout the optimisation process, an external validator verifies hard-constraint feasibility and calculates the associated soft-constraint penalty.

The central aim of this research is to determine where LLM-based reasoning can contribute most effectively within a highly constrained optimisation pipeline, rather than assuming that the LLM should independently solve the entire rostering problem. Specifically, the thesis evaluates whether combining high-level LLM reasoning with deterministic candidate generation and validation can provide a more reliable approach to roster improvement.

The research addresses the following questions:

1. Can a systematic computational framework construct a nurse roster that satisfies all INRC-II hard constraints?
2. Can deterministic and LLM-assisted repair strategies reduce the soft-constraint penalty of a roster that is already hard-feasible?
3. How does the role assigned to the LLM affect the effectiveness of roster improvement?

The principal contributions of this thesis are:

1. A framework for constructing and validating hard-feasible rosters based on the INRC-II constraint model.
2. A systematic comparison of multiple roster-improvement strategies, ranging from deterministic local modifications to different forms of LLM-assisted agentic search.
3. A hybrid optimisation framework in which deterministic procedures enforce constraints and evaluate candidate solutions, while the LLM provides higher-level reasoning and search guidance.
4. An analysis of successful and unsuccessful search behaviour, with particular attention to the effects of hard constraints and interactions among soft-constraint penalties on the improvement of an already feasible roster.

5. An investigation into the appropriate role of LLM agents in constrained combinatorial optimisation, particularly the distinction between direct solution generation and the strategic guidance of deterministic search.

Overall, this thesis conceptualises nurse rostering not merely as a task in which an LLM is instructed to generate a schedule, but as a structured optimisation process in which LLM reasoning, deterministic search, and formal validation perform distinct functions. By separating the construction of a hard-feasible roster from the subsequent optimisation of soft constraints, the research seeks to develop a clearer understanding of both the potential and the limitations of integrating LLM-based agents into highly constrained scheduling problems.

Chapter 2

Literature Review

2.1 Problem Context: Nurse Rostering in Modern Healthcare

The Nurse Rostering Problem (NRP) is an essential challenge that arises in the hospital environment when trying to organize personnel schedules. It is a matter of how to allocate shifts to nurses during the time frame while meeting the requirements of coverage, laws, contractual agreements, and staff preferences. The problem is NP-hard, generally very limited by constraints, and presents difficult situations since constraints of different natures interact strongly (e.g., minimum rest, maximum consecutive shifts, skill requirements for hard constraints, and individual preferences, fairness for soft constraints) [1], [2]. To ensure patient safety and quality of care nurses should be well rested. Booker et al. link the quality of rosters to the detriment of the fatigue risk and the dissatisfaction, to the currently mostly ad-hoc tools and tacit expertise, that poor schedules are a major cause of staff fatigue, and that bad schedules contribute to fatigue risk and dissatisfaction [3]. Holton et al. conducted a similar study in which the findings likewise show that nurses and midwives, when involved in co-designing rostering principles, put forward factors such as equity, transparency, and flexibility as most important [4]. O'Connell et al. examine the pros and cons of electronic and

self-rostering systems and state that the implementation of such systems in reality rather than in operations research (OR) models is challenging owing to factors such as workflow, culture, and trust [5].

Different scenarios and constraints that reflect real-world issues are modelled in a multi-stage nurse rostering problem with weekly updates and historical dependencies that the Second International Nurse Rostering Competition (INRC-II) has defined through a carefully specified file format and validator to establish a benchmark widely used in this field [1].

INRC-II offers real and artificial instances and has become the standard in the NRP literature for comparing different ways of solving the problem. Recently, Mystakidis et al. as an example of hospital-based case studies have demonstrated the use of integer programming models which are hospital-specific and they can be used as a tool to measure how better rosters lead to the hospital's operational efficiency and care quality [6].

2.2 Classical and Hybrid Solution Methodologies for NRP

Ngoo et al. present an exhaustive review of NRP solution methods over the years 2012–2021 that identifies different categories of solution methods such as heuristics, single-solution metaheuristics, population-based metaheuristics, hyper-heuristics, math heuristics, and hybrid approaches [2]. The comparison between those categories of methods indicates a decisive movement towards math heuristics and hybrid metaheuristics, these latter methods resort to a component that is an exact ILP/CP together with local search or metaheuristic frameworks to balance optimality and scalability.

In fact, a handful of recent works clearly show the such an orientation. Turhan and Bilgen came up with a hybrid repair-and-optimize plus simulated annealing strategy where a metaheuristic explores the neighborhood structure while MIP subproblems are used to repair parts of the roster thus the method achieves high-quality results on benchmark instances [7]. Klyve et al. have gone further to the nurse rostering

incorporating the fatigue factor in the model and employing a hybrid algorithm on realistic horizons to solve the problem, thereby demonstrating that it is possible to generate rosters that are not only fatigue-aware but also at scales that are practical [8]. Ceschia et al. solve the NRP issue that is grounded on the realworld nature by employing a simulated annealing method which is tuned to multi-level constraints to achieve high-quality solutions in the operational setting [9].

Mixed-integer programming and math heuristic approaches are still very much in use. Mystakidis et al. illustrate labour regulations, hospital policies, and availability very well through an IP model and prove with the help of modern solvers and by model design that there can be a considerable improvement over the existing practice [6]. Chen et al. present the neural-network–assisted method plus the combined MIP + deep neural network approach for NRP, the one in which a learned model guides a tree search or hybrid method, thereby providing an example of how machine learning can speed up evaluation and search in personnel rostering [10], [11].

As a whole, the NRP corpus of research has evolved from the employment of solely MIP/CP models to the use of complex hybrids and problem-specific metaheuristics. Nevertheless, the majority of solutions still entail a considerable amount of manual labor for designing neighborhoods, penalties, and decomposition strategies and it is still costly to adapt them to new institutions (with new constraints or preferences) [2], [7], [8].

2.3 Machine Learning and Reinforcement Learning for Personnel Scheduling

A parallel research also considers the incorporation of ML and RL methods to lessen the workload of or to automatically perform activities related to rostering and staff scheduling. In their work, Bello et al. presented Neural Combinatorial Optimization with Reinforcement Learning in which a pointer-network policy trained with RL generates solutions for TSP and alike problems, thereby proving that neural policies can

interactively learn from data efficient construction heuristics [12]. Dai et al. suggest Learning Combinatorial Optimization Algorithms over Graphs where graph neural networks and RL are used to learn greedy heuristics for minimum vertex cover, max-cut, and TSP by considering the algorithm as a learnable entity [13].

In a similar manner, Nagayoshi and Tamaki experiment with a Q-learning method targeting nurse work shifts on the top of a constructive nurse scheduling system. They find shiftexchange operations that improve initial rosters and also satisfy practical constraints [14]. Platten et al. examine the potential of RL in the automatization of personnel scheduling in industries with over ten thousand employees and shifts. They prove that RL-driven policies can be schedule proposers or existing ones' improvers in a vast, heterogeneous setting [15]. Also, neural-assisted tree search for personnel rostering is the next step in deep networks where they guide branching decisions and shorten the search space [16].

In context of NRP, Chen et al. first propose a neural-network-assisted NRP method, and then a hybrid MIP + deep neural network model that trains to estimate the evaluation functions or guide the search of candidate rosters [10], [11]. Simultaneously, Hartman et al. and coworkers view ML as a tool for roster evaluation. They employ classifiers to quickly predict acceptance or rejection of a candidate schedule that meets complicated organizational preferences [10], [23].

2.4 LLMs for Optimization and Operations Research

Large Language Models (LLMs) have altered the manner in which optimization models, heuristics, and decision-support systems are created, essentially. LLMs are significantly different from traditional machine learning methods (where models operate only on structured tensors) in that they are capable of processing unstructured textual constraint descriptions, problem statements in natural language, and even reasoning traces of algorithms. So, the interaction between optimization and AI is going to a completely new level.

2.4.1 LLMs as Optimizers via Prompt-Driven Search

The main concept behind this study is to use LLMs as optimization tools which accept prompts as input and, by natural language reasoning, they refine candidate solutions. Yang et al. propose OPRO (Large Language Models as Optimizers) where an optimization problem is given in the form of a text and the LLM suggests better solutions based on a series of already evaluated candidates [19]. OPRO provides evidence that LLMs are capable of gradient-free, derivative-free optimization in a way that they use the linguistic structure of prompts as the optimization mechanism. Various experiments have been carried out to demonstrate that this technique may be used as a competitive tool for symbolic regression, scheduling heuristics, and small mixed-integer problems.

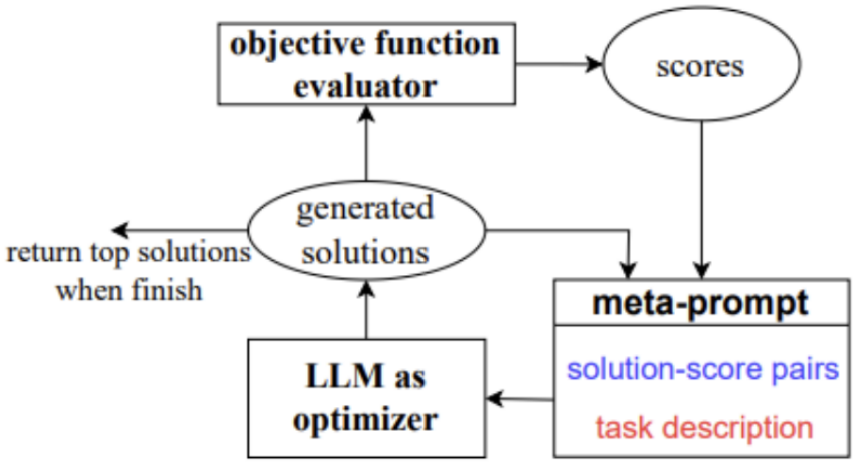


Figure 2.1: An overview of the OPRO framework

Complementary studies consider multi-step refinement mechanisms such as self-consistency, where a number of candidate reasoning paths are generated and combined to provide better solutions, and verbal reinforcement learning, as in Reflexion [27], where an agent looks back at previous outputs in order to not repeat past errors. These kinds of mechanisms exemplify that LLMs can do meta-reasoning regarding the search process itself.

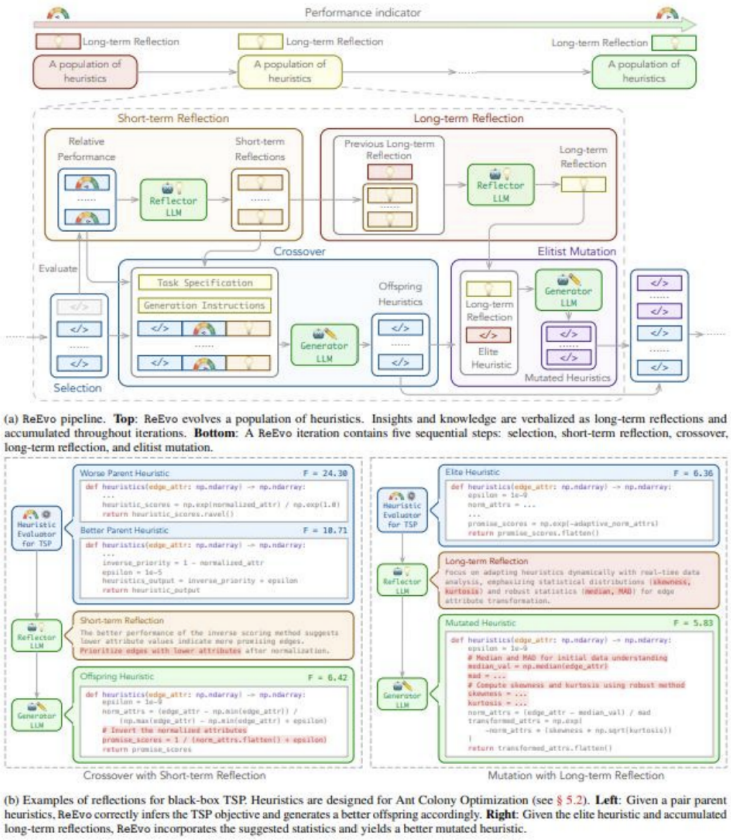
2.4.2 LLMs as Evolutionary Hyper-Heuristics

A more structured approach sees LLMs as one that first generates and then improves heuristics.

Liu et al. present the concept LMEA where a genetic algorithm operation (selection, crossover, mutation) is performed by an LLM on a prompt basis in a way which is entirely descriptive [20].

Ye et al. ReEvo take this concept further by considering heuristics as the descriptions in natural language that change through the input of the reflective feedback [21]. An LLM in ReEvo generates heuristics candidates, a classical solver evaluates these candidates and the LLM feeds the feedback of the evaluation to the next generations. This closedloop system can therefore reasonably be regarded as a meta-heuristic planner to the LLM which can now autonomously find high-performing solutions to combinatorial optimization problems like scheduling, routing and packing.

ReEvo is highly significant to the NRP as it demonstrates the manner in which an LLM can be utilized to create and develop constructive heuristics and thus offer an example of the type of reasoning that we would desire to apply in addressing the complex constraints of NRP and the dynamically changing policies.



(a) ReEvo pipeline. **Top**: ReEvo evolves a population of heuristics. Insights and knowledge are verbalized as long-term reflections and accumulated throughout iterations. **Bottom**: A ReEvo iteration contains five sequential steps: selection, short-term reflection, crossover, long-term reflection, and elitist mutation.

(b) Examples of reflections for black-box TSP. Heuristics are designed for Ant Colony Optimization (see § 5.2). **Left**: Given a pair parent heuristics, ReEvo correctly infers the TSP objective and generates a better offspring accordingly. **Right**: Given the elite heuristic and accumulated long-term reflections, ReEvo incorporates the suggested statistics and yields a better mutated heuristic.

Figure 2.2: An illustration of ReEvo

Conception of EoH (Evolution of Heuristics) by Liu et al. is that LLMs are capable of generating algorithmic "building blocks"–neighborhood definitions, acceptance criteria, repair moves–which can be later converted into the executable code and evaluated in an EA loop [22].

These papers set a background of LLMs being language-based hyper-heuristics that are able to create search strategies for combinatorial problems like TSP, VRP, bin packing, and routing.

Importantly for NRP, these frameworks offer a pathway to automatically create:

- neighborhood structures (e.g., shift swaps, stretch reductions, rest-window repairs)
- constraint-aware penalty functions

- multi-objective trade-off rules.

In contrast to traditional hyper-heuristics—where operators must be manually engineered— LLM-based methods can adaptively generate operators from text describing constraints and objectives.

2.4.3 LLM Code Generation and Algorithmic Improvement

Large language models (LLMs) are also capable of working at the source code level as well, thereby making existing algorithms more efficient.

Sartori and Blum have shown experimentally that when given the code of a meta-heuristic, language models can change it by adding new operators, tuning parameters, or rewriting parts of the algorithm, thus creating new versions that are better and, hence, achieve higher scores in multiple benchmark tests [28].
These experiments lead to the conclusion that LLMs are able to:

- rewrite local-search routines
- restructure evolutionary operators
- tune acceptance criteria and intensification/diversification rules
- directly integrate domain constraints into solver logic.

The code-generation component here is very important for NRP, as it has been traditionally the costliest step to encode hospital-specific rules in algorithmic form

2.4.4 LLMs for Formal Optimization Modelling

Huang et al. come up with ORLM, a full-stack pipeline idea that involves fine-tuning an LLM on a large synthetic dataset (OR-Instruct) made up of natural-language optimization tasks and their corresponding formal models [23]. ORLM is capable of producing:

- decision variables
- constraints
- objective functions
- solver-ready code (e.g., Pyomo/CP-SAT)

Naturally, this is very important for NRP, where the constraint set changes significantly from one hospital to another and is usually stated in policy documents, enterprise agreements, and local guidelines.

Hao et al. come up with the concept of LLMFP (formalized programming), where an LLM identifies entities and relationships in the text, and then creates a correct-by-construction optimization model [24]. LLMFP includes methods for checking that constraints produced are correct from a logical point of view. The main idea is that LLMs are able to connect less structured human policies with more structured solver formulations, thus, in a way, obviating the need for a deep level of operations research knowledge.

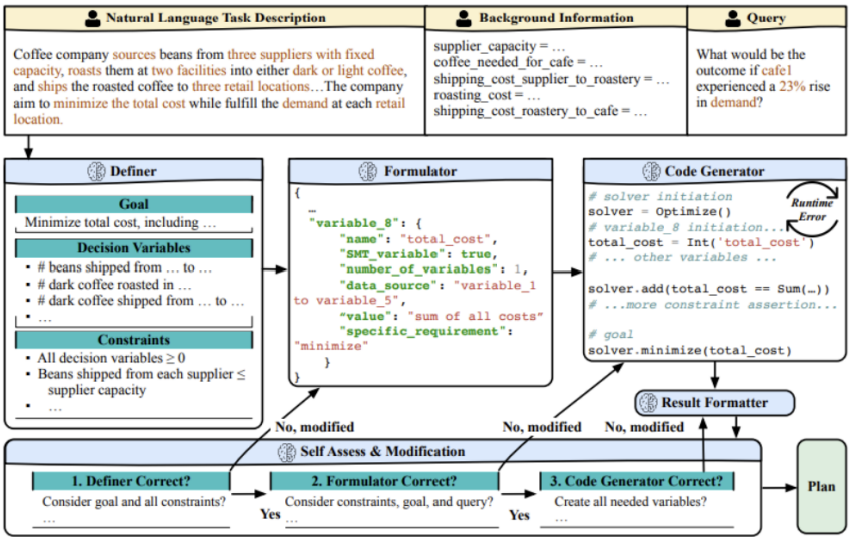


Figure 2.3: An overview of LLMFP

2.4.5 LLMs as End-to-End Combinatorial Optimizers

Some of the recent literature investigate whether LLMs, particularly fine-tuned models, can solve combinatorial problems directly without the using any classic solvers. Abgaryan et al.'s work on LLMs can Schedule [25] shows how fine-tuning can be used to teach LLMs job shop scheduling (JSSP); hence the performance of the models is competitive for small/medium instances. Jiang et al. state that LLMs, when trained with domain-specific prompts, can be used to optimize solution to complex problems without the need for external solvers [26].

Though these models still have difficulties with large-scale OR benchmarks, they stand as a proof-of-concept that LLMs can be used as a single-agent solution for structurally complex scheduling problems.

Chapter 3

Problem Definition and INRC-II Constraints

3.1 Nurse Rostering Problem Formulation

Nurse Rostering Problem (NRP) is a combinatorial optimisation problem that assigns nurses to shifts for a given planning period, taking into account operational and contractual constraints.

Let N denote the set of nurses, D the set of days, S the set of shift types, and K the set of skills. A roster can be formulated using a binary assignment variable.

$$x_{n,d,s,k} = \begin{cases} 1, & \text{if nurse } n \text{ is assigned to shift } s \text{ with skill } k \text{ on day } d, \\ 0, & \text{otherwise.} \end{cases} \tag{3.1}$$

The rostering problem has two types of constraints: hard constraints and soft constraints. The hard constraints denote the feasibility of the rostering problem. If a hard constraint is violated, then the roster is infeasible. The soft constraints represent the desirable aspects of the rostering problem which are desirable scheduling conditions but can be violated. If the soft constraints are violated, then a penalty is associated with each violation of soft constraints. The objective of the rostering problem can

be split into two phases: firstly, a feasible rostering schedule satisfying all hard constraints is obtained. Secondly, the soft constraints are maximally satisfied while keeping the rostering problem feasible. The problem investigated in this thesis is the Second International Nurse Rostering Competition [1].

3.2 INRC-II Problem Structure

The INRC-II problem is the problem formulation of the competition. The new developments of INRC-II are: multi-stage structure; split-shift modeling capability; and constraints between two stages. Each planning period is composed of several stages. Some scheduling constraints in one period depend on the assignments made in previous periods.

For example, the number of continuous working days may affect the total assignment, etc. Thus, a decision of one week may affect the constraint satisfaction and the roster quality in more than one week. For each scheduling period, the roster must first satisfy four hard constraints. Once the feasibility is established, the accumulated penalties associated with the soft constraints are used to evaluate the quality of the roster.

3.3 Hard Constraints

A roster is considered hard-feasible when all hard constraints are satisfied, and infeasible otherwise. If there is a violation of any of the hard constraints, then the soft constraints and their cost are ignored, and the roster is declared as infeasible, irrespective of its soft-constraint cost.

3.3.1 H1 — Single Assignment Per Day

A nurse may work at most one shift on a given day. A nurse therefore cannot be assigned simultaneously to multiple shift types or multiple staffing requirements on the

same day.

This can be represented as

$$\sum_{s \in S} y_{n,d,s} \leq 1 \qquad \forall n \in N,\ d \in D \tag{3.2}$$

This constraint prevents overlapping assignments and ensures that each nurse is treated as a finite daily resource.

3.3.2 H2 — Under-staffing

Every shift and skill combination must contain at least the minimum number of nurses specified by the staffing requirements.

For a required minimum coverage $C^{\min}_{d,s,k}$,

$$\sum_{n \in N} x_{n,d,s,k} \geq C^{\min}_{d,s,k} \tag{3.3}$$

Failure to satisfy minimum coverage makes the roster infeasible.

It is important to distinguish minimum staffing from optimal staffing. Minimum staffing is a hard requirement, whereas failing to reach the optimal staffing level contributes a soft penalty.

3.3.3 H3 — Shift Type Successions

Assignments on consecutive days must follow the legal shift-successions defined by the INRC-II scenario.

For example, if a transition from shift s_1 to s_2 is forbidden, a nurse assigned to s_1 on one day cannot be assigned to s_2 on the following day.

Conceptually,

$$y_{n,d,s_1} + y_{n,d+1,s_2} \leq 1 \tag{3.4}$$

for each prohibited succession (s_1, s_2).

This constraint introduces a temporal dependency into the roster because the feasibility of an assignment depends not only on the current day but also on neighbouring assignments.

3.3.4 H4 — Required Skills

A nurse may only satisfy a staffing requirement when the nurse possesses the skill required for that assignment.

Therefore,

$$x_{n,d,s,k} = 0 \tag{3.5}$$

whenever nurse n does not possess skill k.

This prevents a roster from satisfying staffing numbers using nurses who are not qualified for the corresponding requirement.

3.4 Soft Constraints

The soft constraints, for example, can be violated without making the roster infeasible, but each violation contributes a weighted penalty to the total roster cost. After obtaining a hard-feasible roster, the objective of the optimisation is to minimise the accumulated soft-constraint penalties. In this work, we use the soft constraints of INRC-II, as shown in Table 3.1.

Table 3.1: Soft constraints and penalty weights

Soft Constraint	Brief Description	Weight
Optimal Staffing	Penalises each missing nurse when staffing is below the optimal requirement. Staffing must still remain above the minimum hard requirement.	30
Consecutive Assignments	Penalises violations of the permitted minimum or maximum number of consecutive assignments, including consecutive working days and consecutive assignments of the same shift type.	15 / 30
Consecutive Days Off	Penalises sequences of days off that fall below the minimum or exceed the maximum permitted by the nurse's contract.	30
Preferences	Penalises assigning a nurse to a shift or day that the nurse requested not to work.	10
Complete Weekend	Penalises nurses with a complete-weekend requirement when they work only Saturday or only Sunday instead of both days or neither day.	30
Total Assignments	Penalises the difference when a nurse's total number of assignments falls outside the minimum or maximum specified by their contract.	20
Total Working Weekends	Penalises each working weekend exceeding the maximum number permitted by the nurse's contract.	30

These soft constraints are interconnected, meaning that a roster modification intended to reduce one penalty may introduce or increase another. Consequently, the quality of a candidate roster is determined by its overall weighted penalty, rather than by the improvement of an individual soft constraint in isolation

3.4.1 Objective Function and Roster Cost

Once all hard constraints have been satisfied, roster quality is determined by the weighted sum of soft-constraint violations.

The objective can be expressed generally as

$$C(R) = \sum_{i=1}^{m} w_i V_i(R) \tag{3.6}$$

where:

- R represents the roster,
- $V_i(R)$ represents the number or magnitude of violations of soft constraint i,
- w_i represents the corresponding penalty weight, and
- $C(R)$ represents the total roster cost.

The optimisation objective is

$$\min C(R) \tag{3.7}$$

subject to

$$H_j(R) = \text{satisfied} \qquad \forall j. \tag{3.8}$$

Lower cost is preferred only if the roster is still hard-feasible. A roster which is infeasible because of the hard constraints is not considered to be a "good" solution even if its soft-constraint penalty is very low.
A definition of an improvement is crucial, especially for the optimisation algorithms that we describe later in the thesis. A candidate improvement of a problem instance isn't considered an improvement just because the penalty value of one of its components is reduced. It must preserve all hard constraints and reduce the objective value.

3.5 Feasibility and Validation

A roster is said to be feasible if all hard constraints of INRC-II problem are fulfilled. A violation of any of these constraints makes a roster infeasible, whatever be the penalty of soft constraints. This paper evaluates the quality of the rosters generated by the proposed framework through the INRC-II validator[1]. The INRC-II validator verifies that the roster generated by the proposed framework is feasible i.e. No hard constraint is violated and that the soft constraint violation penalty of the hard-feasible roster is minimal. At two stages in the research, validation is used. First, validation is used during roster construction to make sure that the generated roster is hard-feasible.

Second, validation is used during the improvement stage to evaluate every suggested modification of the roster. A modified roster is accepted only if it is hard-feasible and it results in a lower total penalty of the soft constraints than the current roster.
The acceptance condition can therefore be expressed as

$$\text{Accept}(R_{\text{new}}) \iff \text{HardFeasible}(R_{\text{new}}) \wedge C(R_{\text{new}}) < C(R_{\text{current}}) \tag{3.9}$$

where R_{new} is the modified roster and $C(R)$ represents the total soft-constraint cost of roster R.

Chapter 4

Methodology

4.1 Overview

This thesis uses a two-stage procedure for solving a nurse rostering problem. The first stage is to find a feasible solution for the INRC-II hard constraints; it is used as a start in the second stage where a sequence of optimisation experiments is performed.
The overall process is represented as

$$\begin{aligned}\text{INRC-II Data} \rightarrow \text{Hard-Feasible Roster} &\rightarrow \text{Validation} \rightarrow \text{Roster Improvement} \\ &\rightarrow \text{Validation} \rightarrow \text{Accept/Reject}\end{aligned} \tag{4.1}$$

There's a strict separation of solution generation and solution evaluation. Rosters or roster modifications may be produced by deterministic algorithms, by the LLM, or by a combination of the two, but feasibility and objective value are always assessed independently by the INRC-II validator.

4.2 Hard-Feasible Roster Construction

The first stage of the methodology involves constructing a nurse roster that satisfies all the hard constraints provided by the INRC-II problem. At first, the LLM was

expected to generate a complete roster from the raw INRC-II scenario, history and weekly requirement input data. In this case, the model had to interpret the input data, derive logical facts from it, and reason over the hard constraints to generate the final nurse assignments. Still, this approach failed to consistently generate a complete hard feasible solution as the LLM had to perform multiple logical reasoning operations across the entire planning horizon, interpret the raw input data, and generate a valid roster at the end. Thus, a simple generator, which translates the input data into logical facts, is developed. To reduce such complex analysis, the roster construction task was reformulated as a structured planning and optimization pipeline.

This idea was motivated by Hao et al. [24], who proposed a LLM-based Formalized Programming (LLMFP) framework where LLMs are leveraged to extract key information from planning problems and formally translate them into optimization problems that can be optimally solved via mature optimization tools. In their work, they demonstrated that the separation of the interpretation and formulation of the problem from the optimization itself can lead to more robust and reliable results compared to a LLM that outputs the final plan directly.

The LLM uses the raw INRC-II input data with a prompt describing how to parse the data and output a structured JSON representation of the data. The complete prompt is provided in Appendix. The raw data contains the scenario information, nurse characteristics, contracts, skills, previous roster history, shift types, legal shift successions, and weekly staffing requirements. The purpose of this first stage is to transform the original heterogeneous input into a consistent machine-readable representation while preserving all information required for roster construction.

The first stage can therefore be represented as

$$\text{Raw INRC-II Data} \rightarrow \text{LLM} \rightarrow \text{Structured JSON}.$$

In the second stage, the generated JSON is used as input to an LLM prompt; in this stage, the LLM isn't asked to directly solve for the nurse assignments. It is instead asked to generate a Python optimisation solver using OR-Tools CP-SAT. The structured JSON provides the solver generation prompt with an explicit representation of

the nurses, skills, contracts, shift requirements, previous assignments, and constraint parameters needed to formulate the optimisation problem. The second stage is therefore represented as

$$\text{Structured JSON} \rightarrow \text{LLM} \rightarrow \text{CP-SAT Solver}.$$

The resulting CP-SAT model defines the nurse assignment decision variables and hard constraints described in Chapter 3; these are the one-to-one assignment for each nurse per day, minimum staffing requirements, skill constraints, and legal shift succession. The CP-SAT solver is then executed deterministically in order to try and find a feasible assignment satisfying these hard constraints.

Then the candidate generated by the solver is converted to the required INRC-II solution format and put into the validator. The validator independently checks that all hard constraints are satisfied. In this way, feasibility is completely determined by the formal constraints as defined rather than by the the LLM's own assessment of its generated solution.

This approach separates the responsibilities of the LLM and the optimisation solver. The LLM is primarily responsible for interpreting the raw rostering information, structuring the problem data, and generating the formal optimisation model, while CP-SAT (Constraint Programming with SAT solver) performs the combinatorial search required to construct the roster. The resulting hard-feasible roster is then used as the starting solution for the roster-improvement experiments described in the following sections.

4.3 Roster Improvement Framework

The research consists of finding a feasibility-preserving improvement technique for a roster. Although our input is a hard-feasible roster, our goal is to find an improvement that minimizes a soft-constraint penalty of that roster. We present improvement methods that all use the same repair framework. Based on the cost and the individual soft-constraint penalties of the current roster the validator evaluates the modifications that are least costly while satisfying all the hard constraints.

A repair method then determines if a direct roster modification, set of candidate modifications, or a repair strategy will be used depending on which optimisation approach is being used. Python applies the change(s) to a copy of the current roster and converts the modified roster to the correct INRC-II format. The modified roster is then evaluated using the same validator.

The improvement loop is therefore

$$R_t \rightarrow \text{Penalty Analysis} \rightarrow \text{Repair Method} \rightarrow R' \rightarrow \text{Validator} \rightarrow \text{Accept/Reject} \quad (4.2)$$

where R_t represents the current roster and R' represents a candidate roster. A candidate is accepted only when it satisfies all hard constraints and produces a strictly lower soft-constraint cost.

4.4 Roster Improvement Methods

Different repair methods were implemented to explore different ways of combining LLM reasoning with deterministic search. In these repair methods, the responsibility of LLM is progressively changed from actually generating individual roster modifications to providing some high level guidelines for deterministic search.

4.4.1 Deterministic Single-Move Search

The deterministic single move method is an easy baseline to provide a baseline without using an LLM. Python systematically generates individual roster modifications and then the validator is called for each candidate; all the changes involve a subset of the current roster. The modified roster is validated for structural validity before passing it to the evaluator. Any candidate candidates that violate hard constraints are rejected, while hard-feasible candidates are compared against the current objective cost.

The method therefore follows

$$\text{Python Candidate Generation} \rightarrow \text{Apply Single Move} \rightarrow \text{Validator} \rightarrow \text{Accept/Reject} \tag{4.3}$$

This method provides a deterministic local-search baseline against which the LLM-assisted approaches can be compared.

4.4.2 Direct LLM Improvement

In the direct LLM approach, the LLM is given the current roster along with information about the soft-constraint penalties and is asked to propose explicit roster modifications. The model return one or more move bundles representing the list of assignments to change. Python does not dictate the decision making strategy in this model. Its main task is to parse the proposed moves, apply the moves to the roster, perform preliminary validation, and pass the candidate solution to the validator.

The workflow is

$$\text{Roster} + \text{Penalties} \rightarrow \text{LLM} \rightarrow \text{Move Bundle} \rightarrow \text{Python} \rightarrow \text{Validator}. \tag{4.4}$$

This approach gives the LLM direct control over the actual modifications made to the roster

4.4.3 ReAct Direct Repair

The ReAct direct approach extends direct LLM improvement by introducing an iterative reasoning-action-observation process[29].

The LLM receives the roster at each iteration, relevant penalty information and repair history. The LLM calculates a possible improvement of the roster and proposes a series of changes to the roster. Python applies the changes to the roster and validates the roster.

The validator's result is returned to the LLM as an observation. The observation

includes whether the modification was hard-feasible and whether any soft penalties increased or decreased.

The interaction follows

$$\text{Reason} \rightarrow \text{Action} \rightarrow \text{Validation} \rightarrow \text{Observation} \rightarrow \text{Reason}. \tag{4.5}$$

Unlike the direct LLM method, the model can therefore use feedback from unsuccessful modifications when deciding its next action.

4.4.4 ReAct Strategy-Neighbourhood Repair

The ReAct strategy-neighbourhood method change the role of the LLM to select the region of the search tree or search type to be explored. The LLM receives the current roster, penalty information, validator feedback, and the history of previously attempted moves. Based on this information, it selects a repair neighbourhood or strategy. Python then generates the concrete candidate moves within the selected neighbourhood.

The candidates are scored according to the validator. The results of the search are returned to the LLM as the new observation for the next iteration of search.

4.4.5 SOCIA-Based Repair

The SOCIA-based method [30] adds a more structured analysis phase before the repair strategy selection; only the Task Understanding Agent, Data Analysis Agent, and Feedback Generation Agent are used in this work.
The Task Understanding Agent is responsible for determining the current state of the optimization process and any relevant scheduling objectives. The Data Analysis Agent reviews the roster and validator information to look for patterns of important penalties and potential areas of repair. The Feedback Generation Agent analyzes the results of previous repair attempts and provides information to assist in the decision process.

The resulting information is used to guide the selection of a repair strategy, while deterministic Python components remain responsible for generating and applying concrete candidate modifications.

The workflow is

$$\begin{aligned}\text{Task Understanding} \rightarrow \text{Data Analysis} \rightarrow \text{Strategy Selection} \rightarrow \text{Python Search} \\ \rightarrow \text{Validator} \rightarrow \text{Feedback.}\end{aligned} \tag{4.6}$$

This approach investigates whether explicitly separating different reasoning responsibilities can provide more informed search guidance than a single LLM decision step.

4.4.6 Agentic Block Repair

The agentic block approach is an method, restricting allowable repairs to only selected regions of the roster.
Firstly, it identifies the current solution as well as the block of nurses and days for which the penalty is subject to a penalty or a potential repair opportunity. Then, it takes this restricted region and the relevant penalty and constraints information and asks the LLM to reason over it to identify the best repair.
The LLM proposes modifications within the selected block, after which Python applies and validates them.

The workflow is

$$\text{Penalty Analysis} \rightarrow \text{Select Nurse/Day Block} \rightarrow \text{LLM Repair} \rightarrow \text{Validator.} \tag{4.7}$$

Restricting the action space is intended to reduce the number of assignments the LLM must consider simultaneously while still allowing coordinated changes involving several related assignments.

4.4.7 Operator-Based Search

T he operator-based search method further separates higher level reasoning and lower level roster modification. The LLM does not directly generate the moves that'll be

applied to the roster. Rather, it analyzes the current roster, the penalties for the soft constraints, and the outcomes of previous repairs to find an appropriate **repair operator**. A repair operator is a pre-defined rule of transformation of the rosters. These transformations define the method of reassigning, swapping or changing selected nurse-shifts in the roster to find the relevant subset of neighboring solutions.

The sequence of modification operations to be applied to each candidate is deterministic Python code and is applies that operator to generate. This allows a larger number of candidate roster modifications to be generated while remaining deterministic and reproducible.

The process can be represented as

$$\text{Roster + Penalty Information} \rightarrow \text{LLM Operator Selection} \rightarrow \text{Python Candidate Generation} \rightarrow \text{Validator} \rightarrow \text{Feedback.} \tag{4.8}$$

Candidates generated in each iteration are independently evaluated by the INRC-II validator. Candidates violating hard constraints are rejected. Candidates that satisfy all hard constraints are evaluated with a penalty, a combination of soft-constraint violations. A candidate is only accepted if it is hard-feasible and has lower total penalty than the current roster. Still, the results of the validator can also be sent back to the LLM, which uses them during later iterations of its heuristic.

In this case, the LLM can tell if the operator generated any feasible candidates and also get information on what penalties were affected and if the search has improved. This design decision puts the LLM in control of how the search is run, while Python generates the search candidate modification and the validator validates the modifications.

4.4.8 Compound Repair

In contrast, the compound repair method explores coordinated bundles of roster modifications. The bundle compound consists of a sequence of assignments. The bundle

compound can perform multiple coordinated assignments. A "coordinated" assignment can change several related activities in order to find a better solution. In other words, a coordinated assignment can change a several related assignments so that some costs are temporarily increased but the bundle compound can generate a lower cost.

The process follows

$$R_t \rightarrow \text{Multiple Coordinated Moves} \rightarrow R' \rightarrow \text{Validator} \rightarrow C(R'). \tag{4.9}$$

The method uses iterative strategy selection, operator-specific candidate generation, and validator feedback. Information about previously attempted bundles and their outcomes is retained so that repeated unsuccessful modifications can be avoided in later iterations.

Compound repair therefore expands the search neighbourhood beyond that of the single-move methods and is intended to provide a mechanism for exploring solutions that cannot be reached through an immediately improving individual move.

4.5 Prompt Construction and Search History

For LLM-assisted approaches, prompts are generated dynamically based on the current state of the optimisation process; the information input to the model includes current INRC-II roster data, the current soft-constraint penalties, repair task information, and previous iterations of the algorithm. For iterative methods, a list of previous repair attempts is maintained. This list includes the strategy or move previously proposed, the resulting roster was hard-feasible, whether it was hard-feasible, the cost change, and an indication of whether the penalties increased or decreased after the modification.

The above mentioned history model allows the LLM to make distinction between previous successful and unsuccessful search directions, instead of treating each API call as an independent optimisation call. Moves are requested in a structured format that can be automatically parsed by Python. Moves that don't conform to this format, or that

fail to pass a simple set of validity checks, are dropped before the actual validator can be run.

4.6 Deterministic Candidate Evaluation

All candidate evaluation is performed independently of the LLM. The LLM is never used to determine whether a roster is feasible or whether a proposed modification represents an objective improvement.

For every candidate, Python performs the following sequence:

$$\text{Generate/Receive Candidate} \rightarrow \text{Apply Modification} \rightarrow \text{Export Roster} \rightarrow \text{Run Validator} \rightarrow \text{Parse Result}. \tag{4.10}$$

The validator returns the hard-constraint status and the soft-constraint objective value. The candidate is retained only when it satisfies the acceptance condition defined in Section 4.3.

This separation ensures that all methods are evaluated against the same deterministic criterion regardless of whether the candidate was produced by Python, directly generated by the LLM, or generated from an LLM-selected strategy.

Chapter 5

Results

5.1 Experimental Setup

Three Large Language Models were used on the same INRC-II nurse rostering instance; the same raw INRC-II data, scenario, history and weekly requirements were presented to all three models. The models were then fed through the same pipeline for generating rosters (described in Chapter 4).

Model	**Temperature**	**Max Output Tokens**
claude-opus-4-8	0.2	16000
gpt-5.2	0.2	16000
gemini-3.5-flash	0.2	16000

Table 5.1: Large Language Model Configurations Used in the Experiments

The two instances of the INRC-II of different sizes were used in the experiments of the present study; the first problem instance involved 5 nurses, 2 skills, and 3 shift types over a 4-week planning horizon. The second problem instance involved a larger problem with 21 nurses, 4 skills, and 4 shift types, and covered a 4-week planning horizon. Thus, both problem instances were used to ensure that the present framework performed properly for two problem sizes.

5.2 Hard-Feasible Roster Result

The first stage of the research successfully generated a nurse roster satisfying all the hard constraints defined by the INRC-II problem. The INRC-II validation program evaluated the generated roster. It confirmed that the roster satisfies all the single-assignment, minimum-staffing, shift-succession, and skill-eligibility constraints.

Roster choices used in the simulation of all roster-improvement experiments in the INRC-II problem were based on a computed 100% hard-feasible roster using the hard constraints provided. The hard-feasible roster became the common starting roster for all roster improvement experiments.

5.3 Initial Soft-Constraint Penalty

Although the generated roster satisfied all hard constraints, it still contained violations of several soft constraints. The validator calculated an initial total soft-constraint penalty of

Model	**5-Nurse Penalty**	**21-Nurse Penalty**
claude-opus-4-8	1835	2865
gpt-5.2	1905	2915
gemini-3.5-flash	1920	2890

Table 5.2: Initial Soft-Constraint Penalties for the 5-Nurse and 21-Nurse Instances

These hard-feasible rosters and their corresponding initial penalties were used as the starting points for the subsequent roster-improvement experiments.

5.4 Comparison of Roster-Improvement Methods

Several deterministic and LLM-assisted repair methods were evaluated using the same hard-feasible roster. These methods differed primarily in the role assigned to the LLM

and in the size and structure of the search neighbourhood explored. The prompt templates used for each method are provided in the Appendix.

Method	Claude (Initial: 1835)		GPT-5.2 (Initial: 1905)		Gemini (Initial: 1920)	
	Best Cost	Improvement	Best Cost	Improvement	Best Cost	Improvement
Direct LLM Repair	1805	30	1890	15	1900	20
Deterministic Single Move	1800	35	1870	35	1905	15
ReAct Direct	1835	0	1875	30	1910	10
ReAct Strategy-Neighbourhood	1800	35	1900	5	1890	40
SOCIA-Based Repair	1820	15	1895	10	1920	0
Agentic Block Repair	1795	40	1905	0	1890	30
Operator-Based Search	1830	5	1875	30	1920	0
Compound Repair	1825	10	1905	0	1905	15

Table 5.3: Comparison of roster-improvement methods across the three LLM-generated hard-feasible rosters for 5 nurse instance.

No improvement was observed for the 21-nurse instance.

5.5 Best Results for the 5-Nurse Instance

The best result obtained for each LLM-generated roster is summarised in Table 5.4.

Model	Initial Cost	Best Cost	Best Method	Improve
Claude	1835	1795	Agentic Block Repair	40
GPT-5.2	1905	1870	Deterministic Single Move	35
Gemini	1920	1890	ReAct Strategy-Neighbourhood / Agentic Block Repair	30

Table 5.4: Best roster-improvement results for the 5-nurse instance.

The lowest penalty received from the Claude-generated roster was 1795 using Agentic Block Repair, a saving of 40 from the first penalty of 1835. The lowest penalty, 1870, using deterministic single move has been obtained by the GPT-5.2-generated roster, which is 35 lower than the original cost of 1905. The lowest number of penalties reached was 1795. Both teams ReAct Strategy-Neighbourhood and Agentic Block Repair managed to achieve this result, which means a 30 drop in penalties from the initial amount of 1835.

5.6 Results and the 21-Nurse Instance

The same methods were employed for the larger 21-nurse INRC-II instance.
No improvement in the soft-constraint penalty was observed using any of the methods. The generated candidate modifications were evaluated, but none resulted in a lower total soft-constraint penalty value while preserving all hard constraints. Hence, the cost values of the best candidate modifications obtained for each experiment in the second phase were equal to their corresponding initial hard-feasible roster cost values.

5.7 Summary of Results

For the 5-nurse instance, the deterministic and LLM assisted repairs were able to decrease the penalty of the soft-constraint while preserving feasibility of the hard-constraints. The maximum penalty reduction was 40 for Claude, 35 for GPT-5.2 and 30 for Gemini; for the 21-nurse instance, no improvement was obtained when employing any of the repair methods. All candidate rosters that were not hard or soft-feasible were discarded.

Chapter 6

Discussion

This chapter interprets the experimental results presented in Chapter 5 and relates them to the research questions introduced at the beginning of this thesis. The discussion focuses on the ability to construct hard-feasible rosters, the effectiveness of different repair approaches, the changing role of the LLM within the optimisation pipeline, and the effect of increasing problem size.

6.1 Research Questions

6.1.1 R1: Can a systematic computational framework construct a nurse roster that satisfies all INRC-II hard constraints?

We show that a systematic computational procedure can successfully formulate a hard-feasible nurse roster that satisfies the INRC-II constraint structure. Instead of letting the LLM decide the final nurse schedule, the final solution formulation decoupled the problem interpretation component from the combinatorial optimisation component.

To enable the use of LLM to automatically generate CP-SAT optimisation solvers for roster generation problems, we first have to transform raw INRC-II input to structured

data and then convert this structured representation to a CP-SAT optimisation model. The generated solver is the CP-SAT solver.

Separating the assignment decision from the problem interpretation is important since in the first attempt to make the LLM build a complete roster from the input data it must simultaneously reason about the input data, update the state of the roster, reason about temporal dependencies, and explicitly satisfy all hard constraints.

Thus, the successful construction of hard-feasible rosters suggest that the LLM is more reliable in this stage, when used to assist with problem formalisation, than when solely used to solve the complete combinatorial problem.

6.1.2 R2: Can deterministic and LLM-assisted repair strategies reduce the soft-constraint penalty of a roster that is already hard-feasible?

The results show that deterministic and LLM-assisted repair strategies can reduce soft-constraint penalties of the small 5-nurse instances; however, the effectiveness of these repair strategies was dependent on both the repair approach and the initial hard-feasible roster. For the Claude-generated roster, the initial penalty of 1835 was reduced to 1795, giving a maximum improvement of 40. For GPT-5.2, the initial penalty of 1905 was reduced to 1870, giving an improvement of 35.

The improvements weren't always due to a single best-performing method, either. Agentic Block Repair was the best performing method for Claude, Deterministic Single Move was the best for GPT-5.2, and both ReAct Strategy-Neighbourhood and Agentic Block Repair were the best for Gemini. This suggests that the effectiveness of a repair method depends on the underlying roster structure.

Even if two hard-feasible hard-feasible rosters are generated from the same problem instance, they could have a different pattern of soft-constraint violations. A repair method that is suitable for one penalty scheme may not be as effective when used for

another; results for the larger 21-nurse instance are more ambiguous. None found better solutions compared to the most commonly used repair heuristics with the penalty for violating soft-constraints lowered and the hard-constraints still respected.

6.1.3 R3: How does the role assigned to the LLM affect the effectiveness of roster improvement?

The experiments progressively removed responsibility for roster modification decisions from the LLM. Direct LLM Repair allows the LLM to compute the precise modifications to be made to the roster. ReAct Direct removes the LLM completely and instead uses it to find the remaining (i.e. Not validated) assignments. This returns feedback to the model prior to the next repair attempt. ReAct Strategy-Neighbourhood and Operator-Based Search involve the LLM no longer assigning the location or direction of repair. Instead, the LLM decides the area of the roster to explore or the type of repair to perform.

Python generates the specific candidate roster assignments. The results don't show a simple relation between giving more or less control to the LLM and the quality of the result. Direct LLM Repair improved all three 5 nurse starting rosters, but it didn't produce the best result for any of them. Similarly, high level methods like ReAct-Strategy-Neighbourhood performed well for some starting solutions, but didn't improve as well for others. The best results were obtained by the different divisions of responsibility.

Agentic Block Repair produced the highest result for Claude, and jointly the highest result for Gemini, while a fully deterministic Single Move approach produced the highest result for GPT-5.2. These results show that the value of the LLM isn't necessarily linked to the amount of direct control that it has over roster assignments. Its effectiveness depends on the nature of the search space that it is being used to explore. Direct generation allows the LLM to propose specific changes.

This is an effective approach when all the operations are available and the exact ef-

fect is needed. Strategy and operator selection also allows deterministic methods to perform more widespread exploration of the selected neighbourhood. This means that we suggest that using LLM as one of the elements of a hybrid optimisation methodology is more appropriate than using it as a replacement for a deterministic search and validation strategy.

6.2 Effect of Problem Size

A major difference was observed between the 5-nurse and 21-nurse instances. Multiple repair approaches successfully improved the 5-nurse rosters, whereas no improvement was obtained for the 21-nurse instance.

The 5-nurse instance contains two skills and three shift types over four weeks, while the larger instance contains 21 nurses, four skills and four shift types over the same planning horizon. Increasing the number of nurses, skills and shift types substantially increases the number of possible assignment combinations and therefore the number of possible repair actions.

More importantly, roster assignments are interconnected through both hard and soft constraints. Changing an assignment can affect staffing coverage, shift succession, consecutive working patterns, days off, weekends and contractual assignment limits. As the number of nurses and assignments increases, a repair may therefore influence a larger set of dependent constraints.

The absence of improvement in the 21-nurse instance suggests that the repair neighbourhoods explored in this work were not sufficient to identify an improving hard-feasible solution for the larger search space. This does not demonstrate that no better roster exists; rather, it shows that none of the evaluated methods found such a roster under the experimental search procedure.

The difference between the two instances therefore highlights scalability as an important challenge for repair-based LLM-assisted roster optimisation.

6.3 Effect of Search Neighbourhood

the paper, evaluates different methods that define different neighborhood sizes and structures around the current roster in a real time coreset selection technique for online data streams. Each method is computationally efficient; evaluation is facilitated by a short description of each method and the reporting of running times and core-set sizes. Single move searches are based on changes of smaller size. Agentic Block Repair further extends the concept by allowing multiple nurse and day changes. Operator-based searches are more general by defining a candidate generation operator; the work by Compound Repair is a mix of operators that is evaluated. A complete solution can be made locally optimal by performing several assignment changes in parallel (effectively making the decision problem "larger").

This is significant because a locally optimal assignment modification can be impossible to accept on its own, for instance if a change in one nurse's assignment causes an additional penalty that can only be corrected by changing another assignment. But the results of the experiment didn't show that using a larger search space results in a better result; Compound Repair didn't consistently outperform simpler approaches, and the deterministic Single Move method got the best result for GPT-5.2. This results in a trade-off between a large neighbourhood size and a more effective search. A larger neighbourhood has more opportunities for better solutions. But at the same time, the number of candidate combinations that must be considered during the search increases.

6.4 Interaction Between Soft Constraints

The 5-nurse instance showed that the repair methods were able to reduce the soft-constraint penalty, but the overall improvements remained relatively small. This indicates that once a hard-feasible roster has been obtained, further optimisation becomes difficult because the soft constraints are strongly interconnected.

A modification that reduces one penalty can simultaneously increase another. For

example, changing an assignment to improve staffing coverage or a consecutive work pattern may create additional penalties related to days off, weekend requirements, preferences, or total assignments. As a result, a locally beneficial repair does not necessarily produce a lower overall objective value.

The limited improvement observed for the 5-nurse instance therefore suggests that the repair methods were often able to improve individual parts of the roster, but these gains were partially or completely offset by increases in other soft-constraint penalties. This interaction between penalties makes further reduction of the total roster cost increasingly difficult.

6.5 Limitations

There are some important limitations of the experiment that should be considered when interpreting the results. All of our experiments were run on two INRC-II (nurses with long shifts) instance sizes. The success of the obtained improvements for the smaller 5-nurse problem and the failure to achieve an improvement for the larger 21-nurse problem indicate that results from the smaller instance aren't necessarily generalizable to large-scale rostering problems.

The models try to repair only a fraction of the complete search space. In particular, large neighbourhood search methods don't complete the search in the rosters space. In such cases, a failure to improve doesn't necessarily mean that there is no solution to improve but simply that none was found during the search process.

The behavior of the LLM-assisted method may also depend on the prompt to the model, since we can use the prompt to change the configuration used to train the LLM-assisted. To compare the methodologies, the optimality of the initial hard-feasible roster is dependent on the soft-constraint penalty values, and potentially the underlying distributions of violations.

As a consequence, repairing the same initial hard-feasible roster for different models

may lead to different optimality landscapes. While these limitations are present, the same validation and acceptance criteria were used consistently for all the experiments. This allows us to compare and contrast the performance of different repair approaches.

Chapter 7

Conclusion

Through this thesis, the following findings were observed:

- A systematic LLM-assisted formalisation and CP-SAT framework can successfully generate nurse rosters satisfying all INRC-II hard constraints.
- Deterministic and LLM-assisted repair methods can reduce soft-constraint penalties for the smaller 5-nurse instance while preserving hard feasibility.
- No single repair method consistently performs best across different starting rosters; the effectiveness of a method depends on the structure and penalty distribution of the current roster.
- Increasing the size of the repair neighbourhood does not necessarily result in greater improvement, as changes that reduce one soft penalty may increase another.
- No improvement was obtained for the larger 21-nurse instance, highlighting the increasing difficulty of repair-based optimisation as the problem size and search space increase.

Overall, the results indicate that LLMs are more suitable as components of a hybrid optimisation framework than as standalone nurse-roster optimisers. The LLM can

contribute to problem formalisation, repair reasoning, strategy selection, and search guidance, while deterministic optimisation and validation provide the reliability required to enforce constraints and evaluate solution quality. This division of responsibility provides a practical direction for integrating LLM-based agents into constrained combinatorial optimisation problems.

7.1 Contributions

This thesis contributes:

- A comparison of several repair approaches in which the role of the LLM ranges from direct move generation to higher-level strategy and operator selection.
- An experimental analysis of the effect of problem size, search neighbourhood, and interactions between soft constraints on roster improvement.
- A clearer characterisation of the role of LLMs in nurse rostering: not as replacements for deterministic optimisation, but as reasoning and search-guidance components within a validated optimisation pipeline.

This reframes the use of LLMs in nurse rostering from direct schedule generation towards collaborative optimisation, where language-based reasoning and deterministic search perform complementary roles. The results also show that further work is required to improve scalability and search effectiveness before such approaches can consistently optimise larger rostering instances.

Bibliography

[1] S. Ceschia et al., "Second International Nurse Rostering Competition (INRC-II): Problem Description and Rules," *arXiv preprint arXiv:1501.04177*, 2015.

[2] C. M. Ngoo et al., "A Survey of the Nurse Rostering Solution Methodologies: The State-of-the-Art and Emerging Trends," *IEEE Access*, vol. 10, pp. 56504–56524, 2022.

[3] L. A. Booker et al., "Nurse rostering: understanding the current shift work scheduling processes, benefits, limitations, and potential fatigue risks," *BMC Nursing*, vol. 23, 2024.

[4] S. Holton et al., "Developing nurse and midwife centred rostering principles: A co-design study," *BMC Health Services Research*, 2024.

[5] M. O'Connell et al., "The impact of electronic and self-rostering systems on nurse staffing and work patterns: a review," *J. Clin. Nurs.*, 2024.

[6] A. Mystakidis et al., "Optimizing Nurse Rostering: A Case Study Using Integer Programming to Enhance Operational Efficiency and Care Quality," *Healthcare*, vol. 12, no. 24, 2545, 2024.

[7] A. M. Turhan and B. Bilgen, "A hybrid fix-and-optimize and simulated annealing approaches for nurse rostering problem," *Computers & Industrial Engineering*, vol. 145, 106531, 2020.

[8] K. K. Klyve et al., "Nurse rostering with fatigue modelling," *Health Care Management Science*, vol. 26, pp. 1–16, 2023.

[9] S. Ceschia et al., "Solving a real-world nurse rostering problem by simulated annealing," *Operations Research for Health Care*, vol. 36, 100379, 2023.

[10] Z. Chen et al., "Neural networked-assisted method for the nurse rostering problem," *Computers & Industrial Engineering*, 2022.

[11] Z. Chen et al., "A combined mixed integer programming and deep neural network approach for the nurse rostering problem," *Applied Soft Computing*, 2023.

[12] I. Bello et al., "Neural Combinatorial Optimization with Reinforcement Learning," *arXiv preprint arXiv:1611.09940*, 2016.

[13] H. Dai et al., "Learning Combinatorial Optimization Algorithms over Graphs," in *Proc. NeurIPS*, 2017.

[14] M. Nagayoshi and H. Tamaki, "An approach of exchanging work shifts using reinforcement learning on a constructive nurse scheduling system," *Journal of Robotics, Networking and Artificial Life*, vol. 9, no. 2, pp. 154–158, 2022.

[15] Platten et al., "Automated Personnel Scheduling with Reinforcement Learning," in *RecSysHR 2022 Workshop Proc.*, 2022.

[16] Z. Chen et al., "Neural Networked Assisted Tree Search for the Personnel Rostering Problem," *arXiv preprint arXiv:2010.14252*, 2020.

[17] S. Yao et al., "Multi-objective Evolution of Heuristic Using Large Language Model," *arXiv preprint arXiv:2409.16867*, 2025.

[18] Z. Li et al., "Knapsack RL: Unlocking Exploration of LLMs via Optimizing Budget Allocation," *arXiv preprint arXiv:2509.25849*, 2025.

[19] C. Yang et al., "Large Language Models as Optimizers," *arXiv preprint arXiv:2309.03409*, 2023 (ICLR 2024).

[20] S. Liu et al., "Large Language Models as Evolutionary Optimizers," in *Proc. IEEE Congress on Evolutionary Computation*, 2023; also *arXiv preprint arXiv:2310.19046*, 2023.

[21] H. Ye et al., "ReEvo: Large Language Models as Hyper-Heuristics with Reflective Evolution," in *Proc. NeurIPS*, 2024; also *arXiv preprint arXiv:2402.01145*, 2024.

[22] F. Liu et al., "Evolution of Heuristics: Towards Efficient Automatic Algorithm Design Using Large Language Model," in *Proc. ICML*, 2024; also *arXiv preprint arXiv:2401.02051*, 2024.

[23] C. Huang et al., "ORLM: A Customizable Framework in Training Large Models for Automated Optimization Modeling," *arXiv preprint arXiv:2405.17743*, 2024.

[24] Y. Hao, Y. Zhang, and C. Fan, "Planning Anything with Rigor: General-Purpose Zero-Shot Planning with LLM-based Formalized Programming," *arXiv preprint arXiv:2410.12112*, 2024 (ICLR 2025).

[25] H. Abgaryan, A. Harutyunyan, and T. Cazenave, "LLMs can Schedule," *arXiv preprint arXiv:2408.06993*, 2024.

[26] S. Jiang, M. Xie, and J. Luo, "Large Language Models for Combinatorial Optimization of Design Structure Matrix," *arXiv preprint arXiv:2411.12571*, 2024; extended version in *Proc. Design Society*, 2025.

[27] N. Shinn et al., "Reflexion: Language Agents with Verbal Reinforcement Learning," *arXiv preprint arXiv:2303.11366*, 2023.

[28] C. C. Sartori and C. Blum, "Irace-Evo: Automatic Algorithm Configuration Extended with LLM-based Code Evolution," *arXiv preprint arXiv:2511.14794*, 2025.

[29] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing Reasoning and Acting in Language Models," in *Proc. International Conference on Learning Representations (ICLR)*, 2023.

[30] Y. Hua, S. Weatherhead, M. Jafari, H. Xue, and F. D. Salim, "SOCIA-EVO: Automated Simulator Construction via Dual-Anchored Bi-Level Optimization," *arXiv preprint arXiv:2604.17351*, 2026.

Appendix 1

A.1 Data Parsing Prompt

```
You are a STRICT data parser for an INRC-style Nurse Rostering Problem.

TASK:
Convert the input into COMPLETE structured JSON for a multi-week nurse rostering model.
     Return only the json that doesn't give error with json.load().

IMPORTANT RULES:

- Return ONLY valid JSON
- Do NOT summarize
- Do NOT skip weeks
- Do NOT leave fields empty if data exists
- Expand ALL coverage to explicit day-level entries (W1_Mon ... Wn_Sun)
- Preserve BOTH structured data AND original constraint wording
- All JSON strings must be properly escaped
- Do NOT include markdown, code fences, comments, or explanation outside JSON

NORMALIZATION RULES

- Normalize week/day names to:
  - Week1 Monday -> W1_Mon
  - Week1 Tuesday -> W1_Tue
  - ...
  - Week4 Sunday -> W4_Sun
- Normalize shift names to:
  - Early -> E
  - Late -> L
  - Night -> N
  - Day -> D
- Preserve skill names exactly as given in the input
- Preserve nurse names exactly as given in the input

NURSE REQUEST RULE (CRITICAL)

- ALL nurse requests MUST be stored ONLY in "preferences"
- DO NOT create a "leave" field unless the input explicitly states confirmed or
    mandatory leave
- The value "Any" means the nurse requests NOT to work any shift that day
- Preferences are NOT hard constraints
```

```
CONTRACT MAPPING RULE

- If nurses are labeled by contract type such as FullTime or PartTime, map each nurse
    to the correct contract values
- Store final resolved per-nurse contract values in "contracts"

CONSTRAINT PRESERVATION RULE

- Extract ALL constraints EXACTLY as written
- Store them in:
  "constraints_text": [ ... ]
- Do NOT paraphrase or summarize
- Preserve line content faithfully, but ensure JSON validity by escaping quotes where
    needed

OUTPUT FORMAT (STRICT JSON)

{
  "sets": {
    "N": [list of nurses],
    "D": [ALL days across ALL weeks],
    "S": [shift types],
    "K": [skills]
  },

  "parameters": {
    "coverage": {
      "W1_Mon": {
        "E": {
          "HeadNurse": {"min": X, "opt": Y},
          "Nurse": {"min": X, "opt": Y}
        },
        "L": {
          "HeadNurse": {"min": X, "opt": Y},
          "Nurse": {"min": X, "opt": Y}
        },
        "N": {
          "HeadNurse": {"min": X, "opt": Y},
          "Nurse": {"min": X, "opt": Y}
        }
      }
    },

    "qualifications": {
      "nurse": {
        "skill": 0
      }
    },

    "contracts": {
      "nurse": {
        "min_total": int,
        "max_total": int,
        "min_consec_work": int,
        "max_consec_work": int,
        "min_consec_off": int,
        "max_consec_off": int,
```

```
      "max_weekends": int,
      "complete_weekend": 0
    }
  },

  "shift_type_limits": {
    "shift": {
      "min_consec": int,
      "max_consec": int
    }
  },

  "shift_successions": {
    "forbidden": [["shift1", "shift2"]]
  },

  "historical_assignments": {
    "nurse": {
      "lastAssignedShiftType": string,
      "numberOfConsecutiveAssignments": int,
      "numberOfConsecutiveWorkingDays": int,
      "numberOfConsecutiveDaysOff": int,
      "numberOfAssignments": int,
      "numberOfWorkingWeekends": int
    }
  },

  "preferences": [
    {
      "nurse": "N",
      "day": "D",
      "shift": "S or Any"
    }
  ],

  "soft_rule_weights": {
    "optimal_coverage": int,
    "preferences": int,
    "consecutive_shift_assignments": int,
    "consecutive_working_days": int,
    "consecutive_days_off": int,
    "complete_weekend": int,
    "total_assignments": int,
    "total_working_weekends": int
  }
},

"constraints_text": [
  "exact wording of constraints from input"
],

"hard_rules": {
  "single_assignment_per_day": true,
  "minimum_coverage": true,
  "shift_successions": true,
  "consecutive_shift_limits": true,
  "consecutive_working_days_limits": true,
```

```
    "historical_boundary_conditions": true,
    "skill_compatibility": true
  }
}

VALIDATION REQUIREMENTS

- ALL days across ALL weeks must appear in "D"
- ALL days across ALL weeks must appear in "coverage"
- ALL nurses must appear in "qualifications"
- ALL nurses must appear in "contracts"
- ALL nurses must appear in "historical_assignments"
- ALL preference entries must be preserved in "preferences"
- ALL weights explicitly present in the input must be preserved in "soft_rule_weights"
- If a constraint appears in the input, it must also appear in "constraints_text"
- Do NOT invent missing data
- Do NOT omit available data

Now convert the following input:

<<<DATA>>>
{data}
```

A.2 CP-SAT Prompt

```
    You are an expert OR-Tools CP-SAT scheduling engineer.

TASK:
Generate FULLY RUNNABLE Python code to solve the COMPLETE multi-week INRC-style nurse
    rostering problem.

STRICT OUTPUT RULES

Return ONLY Python code and nothing else.

Do NOT output:
- JSON
- markdown fences
- explanations
- headings
- notes before the code
- notes after the code
- pseudocode

Your response must:
- begin with Python code on the first line
- end with the final line of Python code
- be directly saveable as a file named solver.py
- run with: python solver.py

If anything is approximate, assumed, or cannot be modeled exactly:
- include that information only as Python comments inside the code
- do NOT write prose outside the code
```

```
Environment:
- Python 3.14.4
- ortools== 9.15.6755

Compatibility rules:
- Use only APIs available in this exact OR-Tools version
- Do NOT use cp_model.BoolVar in Python
- Treat BoolVars as IntVar-compatible in Python CP-SAT
- Avoid version-fragile type checks

VALIDITY REQUIREMENTS
The generated code is INVALID if any of the following occurs:
- any hard constraint listed below is omitted
- H3 shift successions is not explicitly implemented with model.Add constraints
- historical boundary conditions are omitted from H3 and consecutive constraints
- the model does not use x[n,d,s,k] as the primary assignment variable
- the model allows one worked shift to satisfy more than one skill demand
- the code claims a rule is implemented but does not explicitly add the corresponding
    constraints
- the code reads the JSON but ignores fields required for modeling
- the code does not explicitly audit H3 internally after solving
- the code does not export the final solution in weekly INRC2 JSON format
- the code prints anything other than one short final completion message
- the code does not save weekly solution files under a folder named initial_roster
- the code does not name the weekly files exactly as week0.json, week1.json, week2.json
    , ...

IMPORTANT RUNTIME OUTPUT RULE

The generated solver code MUST NOT print:
- roster output
- debug output
- penalty summaries
- coverage summaries
- nurse summaries
- H3 audit details
- exported JSON contents
- intermediate progress
- solver logs

The code must print ONLY ONE short final message after execution finishes successfully,
     such as:
- Code ran
- Done
- Solver finished

If solving fails or no solution is found, the code may print ONE short failure message
    only, such as:
- No feasible solution
- Solver failed

No other stdout output is allowed.

IMPORTANT RULES

- Use ALL input data
```

```
- Build ONE integrated model covering ALL weeks together
- ALL hard constraints MUST be enforced
- ALL soft constraints listed below MUST be modeled as penalties unless mathematically
    impossible from the given schema
- Do NOT omit any variable, constraint, or penalty required by the data
- Do NOT silently simplify, weaken, or reinterpret any constraint
- Do NOT claim a rule is implemented unless it is explicitly modeled in the code
- If any input field is intentionally not used, explain it only in Python comments
    inside the code
- The code must load input JSON from a file named "response1.json"
- Do NOT hardcode the instance data inside the Python script

PREFERENCE INTERPRETATION (CRITICAL)

Preferences are shift-OFF requests, NOT preferred working shifts.

Interpretation:
- If shift == "Any":
  - the nurse prefers NOT to work ANY shift that day
  - penalty if assigned to any shift that day
- If shift == specific shift s:
  - the nurse prefers NOT to work that specific shift
  - penalty if assigned to that shift only

PRIMARY DECISION VARIABLES

Use the following primary binary decision variable:

x[n,d,s,k]  {0,1}

Meaning:
- nurse n works shift s on day d while covering skill k

IMPORTANT:
- Each worked assignment covers exactly ONE skill
- Do NOT allow multi-skill counting
- Do NOT use x[n,d,s] as the main assignment variable
- Coverage must be satisfied by summing x[n,d,s,k] over nurses for each exact skill k

MANDATORY LINKING RULES

You MUST explicitly define and use:

1. shift_work[n,d,s] = sum_k x[n,d,s,k]
2. work[n,d] = sum_s shift_work[n,d,s]
3. off[n,d] = 1 - work[n,d]

These linking relations MUST be explicitly enforced in the model and then reused by
    later constraints.

HARD CONSTRAINTS

H1. Single assignment per day
For every nurse n and day d:

sum_{s,k} x[n,d,s,k] <= 1
```

```
This MUST be enforced explicitly.

H2. Minimum coverage
For every day d, shift s, skill k:

sum_n x[n,d,s,k] >= coverage[d][s][k].min

This MUST be enforced explicitly.

H3. Shift type successions
THIS IS A HARD CONSTRAINT AND MUST BE EXPLICITLY CODED WITH model.Add(...)

Use the derived binary variables shift_work[n,d,s].

Let forbidden_pairs = shift_successions["forbidden"].

For every nurse n,
for every consecutive pair of planning days (d_prev, d_curr),
for every forbidden pair (a,b) in forbidden_pairs,
you MUST enforce:

shift_work[n,d_prev,a] + shift_work[n,d_curr,b] <= 1

Historical boundary for H3 is MANDATORY and MUST NOT be omitted:
Let first_day = D[0].
Let last_shift = historical_assignments[n]["lastAssignedShiftType"].

If last_shift is not "None", then for every forbidden pair (a,b):
- if a == last_shift,
  you MUST enforce:

shift_work[n,first_day,b] == 0

IMPORTANT FOR H3:
- Do NOT convert H3 into a soft penalty
- Do NOT approximately enforce it
- Do NOT rely on implied legality
- Do NOT skip forbidden pairs
- Do NOT skip the historical boundary
- Do NOT replace this with a postprocessing check only
- The final code MUST contain explicit loops over forbidden pairs and consecutive days
- The final code MUST internally validate H3 after solving before writing output files

H4. Skill compatibility
For every nurse n, day d, shift s, skill k:
If qualifications[n][k] == 0, then:

x[n,d,s,k] == 0

This MUST be enforced explicitly.

IMPORTANT:
- A multi-skilled nurse still covers only one chosen skill on a worked shift
- Do NOT count one assignment toward multiple skill requirements
- Do NOT write coverage as sum(x[n,d,s] * qualifications[n][k]) because that is invalid
     for multi-skilled nurses
```

```
SOFT CONSTRAINTS

All soft constraints below MUST be modeled as penalty terms in the objective, unless
    mathematically impossible from the given schema. If something cannot be modeled
    exactly from the provided data, explain why in Python comments inside the code.

S1. Optimal coverage (weight 30)
For every day d, shift s, skill k:
- penalize shortage relative to coverage[d][s][k].opt
- extra staffing above optimal is not penalized

S2a. Consecutive shift assignments (weight 15)
Respect minimum and maximum consecutive assignments per shift type using
    shift_type_limits.
This evaluation MUST include historical boundary data.

S2b. Consecutive working days (weight 30)
Respect contract min/max consecutive working days.
This evaluation MUST include historical boundary data.

S3. Consecutive days off (weight 30)
Respect contract min/max consecutive days off.
This evaluation MUST include historical boundary data.

S4. Preferences (weight 10)
Interpret preferences exactly as undesired assignments as specified above.

S5. Complete weekend (weight 30)
For nurses whose contract has complete_weekend = 1:
- penalize working exactly one of Saturday or Sunday
- no penalty if both are worked
- no penalty if both are off

S6. Total assignments (weight 20)
For each nurse:
- penalize deviation below min_total
- penalize deviation above max_total

S7. Total working weekends (weight 30)
For each nurse:
- a weekend counts as worked if the nurse works at least one of Sat or Sun
- penalize any excess above contract max_weekends

OBJECTIVE

Minimize the total weighted penalty over all soft constraints.

The code MUST:
- build explicit penalty variables wherever needed
- compute a single total objective expression
- minimize it with model.Minimize(...)

INPUT REQUIREMENTS

The code MUST load JSON from:
response1.json
```

```
The code MUST read and use data from at least these sections when applicable:
- sets
- parameters.coverage
- parameters.qualifications
- parameters.contracts
- parameters.shift_type_limits
- parameters.shift_successions
- parameters.historical_assignments
- parameters.preferences
- parameters.soft_rule_weights

If any field exists but is not usable for exact modeling, mention that only as Python
    comments inside the code.

IMPLEMENTATION REQUIREMENTS

The generated Python code must:
- import OR-Tools CP-SAT correctly
- parse response1.json safely
- build all variables explicitly
- add all hard constraints explicitly
- add all soft-penalty components explicitly
- minimize objective
- solve model
- not print any detailed results
- export weekly solution JSON files in INRC2 format into a folder named initial_roster

REQUIRED MODELING STYLE

The code MUST:
- use descriptive variable names
- preserve clear separation between hard constraints and soft penalties
- use dictionaries keyed by tuples for decision variables where appropriate
- avoid broken CP-SAT API patterns
- avoid Python expressions that misuse max(), min(), abs(), or boolean tests directly
    on CP-SAT linear expressions
- linearize all nontrivial penalty expressions correctly
- never use a CP-SAT expression where a Python boolean is expected
- never write logic that depends on the truthiness of linear expressions
- ensure all sums are over existing indexed variables only

MANDATORY H3 IMPLEMENTATION PATTERN

The generated code MUST contain logic equivalent to the following structure, using the
    actual model variables and data:

for n in nurses:
    for di in range(len(days) - 1):
        d_prev = days[di]
        d_curr = days[di + 1]
        for a, b in forbidden_pairs:
            model.Add(shift_work[n,d_prev,a] + shift_work[n,d_curr,b] <= 1)

for n in nurses:
    first_day = days[0]
    last_shift = historical_assignments[n]["lastAssignedShiftType"]
    if last_shift != "None":
```

```
        for a, b in forbidden_pairs:
            if a == last_shift:
                model.Add(shift_work[n,first_day,b] == 0)

This exact logic may be written with different variable names, but the same meaning
    MUST be present.
If this logic is absent, the answer is invalid.

INRC2 SOLUTION EXPORT REQUIREMENTS (CRITICAL)

After solving, the code MUST export the final roster into INRC2 weekly solution JSON
    objects/files.

The code MUST generate one JSON solution per week in the exact structure:

{
  "scenario": "<scenario_name>",
  "week": <week_number>,
  "assignments": [
    {
      "nurse": "<nurse_name>",
      "day": "<Mon|Tue|Wed|Thu|Fri|Sat|Sun>",
      "shiftType": "<Early|Late|Night>",
      "skill": "<skill_name>"
    }
  ]
}

WEEKLY EXPORT RULES
- One JSON object/file per week
- Use zero-based exported week numbering:
  - W1 -> week 0
  - W2 -> week 1
  - W3 -> week 2
  - W4 -> week 3
- The code MUST infer week number from day labels such as W1_Mon, W2_Tue, etc.
- The day field in exported assignments MUST be only:
  Mon, Tue, Wed, Thu, Fri, Sat, Sun
- The nurse field must be the nurse name
- The skill field must be the assigned skill k from x[n,d,s,k]
- Include ONLY worked assignments, not OFF days

SHIFT NAME MAPPING RULES
The exported INRC2 solution MUST map:
- E -> Early
- L -> Late
- N -> Night

SCENARIO NAME RULES
- The exported JSON MUST contain a scenario field
- If response1.json contains an explicit scenario name, use it
- Otherwise derive it if clearly available
- Otherwise use a clearly documented Python fallback such as "unknown_scenario"
- Do NOT omit the scenario field

FILE OUTPUT RULES
- The code MUST create a folder named:
```

```
  initial_roster
- All weekly JSON solution files MUST be written inside that folder
- The weekly files MUST be named exactly:
  week0.json
  week1.json
  week2.json
  week3.json
  ...
- Do NOT use names like Sol-<scenario>-0.json
- If the instance has N planning weeks, the code must produce exactly N files:
  initial_roster/week0.json through initial_roster/week(N-1).json

MANDATORY WEEKLY GROUPING LOGIC
The code MUST:
- parse each solved assignment day label like W3_Fri
- extract:
  - week index
  - day name
- group assignments by week
- sort assignments within each week in a stable readable order, preferably by:
  1. nurse
  2. calendar day order Mon..Sun
  3. shift type

FAIL-SAFE REQUIREMENTS

If the solver returns infeasible or no solution:
- do NOT fabricate roster files
- print only one short failure message
- do NOT print extra debugging info to stdout

FINAL CHECKLIST THE GENERATED CODE MUST SATISFY

Before finalizing the answer, ensure the produced code:
- uses x[n,d,s,k]
- defines shift_work, work, off
- enforces H1 explicitly
- enforces H2 explicitly
- enforces H3 explicitly across all consecutive days
- enforces H3 historical boundary explicitly from lastAssignedShiftType to first day
- enforces H4 explicitly
- uses exact-skill coverage counting only
- models S1 through S7 as penalties
- uses historical data in consecutive-rule evaluation
- internally audits H3 after solving
- reads response1.json
- exports one INRC2 JSON solution per week
- maps E/L/N to Early/Late/Night
- writes only worked assignments into assignments[]
- creates the folder initial_roster
- writes files named week0.json, week1.json, week2.json, ...
- prints only one tiny final message
- is directly runnable with python solver.py
- returns only Python code and nothing else

The answer will be rejected if H3 is described in comments but not enforced by actual
    model.Add(...) constraints.
```

```
The code MUST include a dedicated clearly labeled section:
# H3 Shift Successions (Hard Constraint)

and a dedicated clearly labeled section:
# H3 Internal Post-Solve Audit

and a dedicated clearly labeled section:
# INRC2 Weekly Solution Export

NOW PROCESS INPUT

<<<DATA>>>
{data}
```

A.3 Direct LLM Method Prompt

```
    You are an expert nurse-rostering local-search assistant.

You are given:
1. the full instance JSON
2. the CURRENT roster as a nested dictionary
3. the COMPLETE raw validator output for the CURRENT roster
4. the results of bundles attempted in previous iterations

The CURRENT roster is always the roster produced by the most recently accepted bundle.
    Do not reason from an older roster found in the search history.

PRIMARY OBJECTIVE

Your only objective is to reduce total cost while preserving all hard constraints.

Use the complete validator output to identify the largest current penalty categories.
    Use the instance, roster, contracts, qualifications, history and assignments to
    infer which nurses, days and patterns are likely responsible.

Default penalty priority:
1. Total assignment constraints
2. Consecutive constraints
3. Non working days constraints
4. Max working weekend
5. Complete weekends
6. Optimal coverage constraints
7. Preferences

Treat this as a default priority only. Target the categories that actually have non-
    zero and materially large costs in the current validator output.

PREVIOUS-RESULT RULES

The previous bundle results may contain these statuses:
- precheck_failed: Python rejected the move structure or roster-level preconditions
    before validation
```

```
- validator_error: the candidate could not be evaluated
- hard_infeasible: the validator found a hard-constraint violation
- feasible_not_improving: hard-feasible but total cost did not decrease
- improving_candidate: improved but was not the best candidate in that iteration
- accepted: selected as the new current roster

Use these results to avoid repeating failed or non-improving bundles. A previous bundle
     may be reconsidered only when the CURRENT roster has changed enough that its
    outcome could now be different.

CRITICAL OUTPUT RULES

Return ONLY a strict JSON array.
Do NOT output markdown.
Do NOT output commentary.
Do NOT output explanations outside JSON.
Do NOT output keys other than those explicitly allowed below.

Each bundle object must contain exactly:
- bundle_id
- reason
- moves

Each moves array must contain 1 to {{MAX_MOVES_PER_BUNDLE}} move objects.
Return at most {{MAX_BUNDLES}} bundles.
The top-level value must be a JSON array.

BUNDLE CONSTRUCTION PROCEDURE

For every proposed bundle:
1. Select one primary current penalty category.
2. Identify a specific nurse, day or sequence that plausibly contributes to it.
3. Check contract limits, qualifications, current workload, adjacent shifts, weekends
    and coverage.
4. Select the smallest move bundle likely to reduce the penalty.
5. Mentally apply all moves sequentially. Every later move must be valid against the
    roster produced by earlier moves.
6. Reject the bundle if it is likely to create minimum-coverage loss, skill-coverage
    loss, double assignment, illegal succession or a larger penalty elsewhere.
7. Reject near-duplicates unless they target meaningfully different opportunities.

The reason must name the primary penalty category and briefly state why the selected
    nurse/day pattern is relevant.

MOVE TYPES

Only these move types are allowed:
- reassign
- swap
- coverage_fill
- weekend_repair

1. REASSIGN

Allowed keys exactly:
- type
- day
```

```
- from_nurse
- to_nurse
- shift
- skill

Format:
{
  "type": "reassign",
  "day": "W2_Fri",
  "from_nurse": "Andrea",
  "to_nurse": "Sara",
  "shift": "L",
  "skill": "Nurse"
}

Rules:
- the source nurse must currently have exactly the stated shift and skill on that day
- the destination nurse must be OFF that day
- the destination nurse must be qualified for the skill
- the move must preserve hard coverage and avoid illegal succession

2. SWAP

Allowed keys exactly:
- type
- day
- nurse_a
- nurse_b

Format:
{
  "type": "swap",
  "day": "W2_Fri",
  "nurse_a": "Andrea",
  "nurse_b": "Sara"
}

Rules:
- do not include from_nurse, to_nurse, shift or skill
- a swap may exchange two working assignments or one working assignment with one OFF
    assignment
- both nurses cannot be OFF
- each nurse must be qualified for the assignment they receive
- the swap must avoid illegal succession and preserve hard coverage

3. COVERAGE_FILL

Allowed keys exactly:
- type
- day
- shift
- skill
- to_nurse

Format:
{
  "type": "coverage_fill",
```

```
  "day": "W3_Fri",
  "shift": "E",
  "skill": "Nurse",
  "to_nurse": "Patrick"
}

Rules:
- the nurse must be OFF and qualified
- use only when the validator shows an optimal-coverage penalty and the nurse is not
    already overloaded
- do not use this to repair an over-assignment penalty
- ensure it does not create illegal succession, excessive consecutive work or weekend
    pressure

4. WEEKEND_REPAIR

Allowed keys exactly:
- type
- nurse
- week

Format:
{
  "type": "weekend_repair",
  "nurse": "Sara",
  "week": 2
}

Exact code behavior:
- the nurse must currently work exactly one of Saturday or Sunday in the selected zero-
    based week
- the operation removes that one existing weekend assignment
- it does not add the missing weekend day
- use it only when removing that assignment is unlikely to break minimum or skill
    coverage and is likely to reduce weekend-related penalties

Do not include day, from_nurse, to_nurse, shift or skill.
The week value must be an integer such as 0, 1, 2 or 3.

DAY LABEL RULES

For reassign, swap and coverage_fill, use labels such as:
- W1_Mon
- W2_Fri
- W3_Sat

Do not use plain day names.

WORKLOAD RULES

Before targeting total-assignment penalties, infer from contracts, history and the
    current roster whether each relevant nurse is over-assigned or under-assigned.

For over-assignment:
- prefer moving an existing assignment to an under-assigned qualified OFF nurse

For under-assignment:
```

```
- prefer receiving an existing assignment through reassign or swap
- use coverage_fill only when it also addresses a genuine optimal-coverage shortage

STRICT NEGATIVE RULES

Do not invent move types.
Do not add extra keys.
Do not propose a move based on a roster state that existed before the latest accepted
    bundle.
Do not repeat a previously failed bundle without a roster-dependent reason.
Do not return malformed JSON.
Do not return a JSON object at the top level.

INSTANCE JSON:
{{INSTANCE_JSON}}

CURRENT ROSTER DICT:
{{CURRENT_ROSTER}}

FULL VALIDATOR OUTPUT FOR CURRENT ROSTER:
{{VALIDATOR_OUTPUT}}

PREVIOUS BUNDLE RESULTS:
{{SEARCH_HISTORY}}
```